Fundamente de Inginerie Software – Subiecte și răspunsuri-model

Notă: Răspunsurile-model sunt orientative și sunt formulate la nivelul unui răspuns scurt de examen, potrivit pentru aproximativ 6 minute de lucru pe subiect.

Subiectul 1

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. La construcția unui sistem software, proiectul a fost conceput astfel încât să fie dezvoltat de 20 de programatori și 10 testeri. În primele 4 luni au lucrat constant 15 programatori și 12 testeri, dar proiectul este în întârziere. În acest context, Legea lui Brooks ne spune că nu putem adăuga oameni noi la proiect fără a-l afecta negativ. Dacă apreciați că afirmația este „adevărată", explicați pe ce aspect al Legii lui Brooks vă bazați afirmația; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Legea lui Brooks spune că adăugarea de oameni la un proiect software deja întârziat tinde să îl întârzie și mai mult, din cauza timpului de integrare, instruire și comunicare. Totuși, nu spune că orice adăugare de personal este imposibilă sau automat negativă. În exemplu, proiectul fusese planificat pentru 20 de programatori, dar au lucrat doar 15, deci este probabil să existe muncă paralelizabilă, iar dacă noii oameni pot fi integrați eficient proiectul ar putea fi accelerat.

Subiectul 2

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Brooks ne spune, în regula cunoscută ca „The Mythical Man-Month", că în ecuația Effort = Man (Person) × Month, cei doi termeni, Man și Month, nu sunt interschimbabili. De exemplu, dacă avem un sistem gândit să fie dezvoltat de 8 oameni în 3 luni, acesta nu va putea fi realizat cu mai puțini oameni, chiar dacă mărim timpul alocat proiectului, de exemplu cu 6 oameni în 4 luni. Dacă apreciați că afirmația este „adevărată", explicați pe ce aspect al Legii lui Brooks vă bazați afirmația; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Brooks critică ideea că oamenii și lunile sunt interschimbabile mecanic, ca într-o simplă înmulțire. Totuși, exemplul din enunț trage concluzia greșită că un proiect de 8 oameni timp de 3 luni nu ar putea fi făcut de 6 oameni în 4 luni. În realitate, depinde de natura sarcinilor: unele sunt secvențiale, altele pot fi paralelizate, iar schimbarea numărului de oameni modifică și costurile de comunicare.

Subiectul 3

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Printr-o reproiectare corespunzătoare, implementarea unui sistem software poate acomoda o echipă de dezvoltare mai mare decât fusese planificat inițial, desigur, în limita constrângerilor secvențiale ale diferitelor tipuri de activități. Dacă apreciați că afirmația este „adevărată", explicați de ce este posibil acest lucru și ce limite apar din activitățile secvențiale; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este adevărată. Prin reproiectare poți împărți sistemul în componente mai independente, cu interfețe clare, astfel încât mai multe persoane sau echipe să poată lucra în paralel. Brooks subliniază totuși că această paralelizare are limite, pentru că unele activități trebuie făcute într-o anumită ordine. De aceea, o echipă mai mare poate ajuta doar dacă arhitectura și organizarea proiectului reduc dependențele și comunicarea inutilă.

Subiectul 4

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Brooks ne spune că diferența dintre un program și un sistem software integrat („Programming Systems Product") este aceea că, într-un sistem software integrat, avem nevoie de mai multă testare și documentație, iar transformarea unui program într-un astfel de sistem poate ajunge chiar să dubleze costurile inițiale. Dacă apreciați că afirmația este „adevărată", explicați ce costuri suplimentare apar atunci când trecem de la un program simplu la un sistem software integrat; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Brooks nu reduce diferența dintre un program simplu și un „Programming Systems Product" doar la mai multă testare și documentație. Mai apar cerințe de integrare, generalizare, interfețe stabile, fiabilitate și mentenabilitate. În plus, costul nu este prezentat ca o simplă dublare; în modelul lui Brooks, un produs software integrat poate ajunge să coste de ordinul de mărime de aproximativ nouă ori mai mult decât un program simplu.

Subiectul 5

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. În procesul de dezvoltare Scrum, „Burndown Chart” ne arată în fiecare moment efortul consumat în Sprintul curent, exprimat prin task-uri încheiate, și îl raportează la Sprintul precedent, pentru a ne da o măsură a productivității Sprintului curent. Dacă apreciați că afirmația este „adevărată”, spuneți ce detalii din definiția „Burndown Chart” vă susțin afirmația; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Un Burndown Chart arată de obicei munca rămasă în Sprint în funcție de timp, nu efortul deja consumat și nici doar task-urile încheiate. El nu are ca scop principal compararea cu Sprintul precedent. Graficul este folosit pentru a vedea dacă echipa se îndreaptă spre terminarea muncii planificate până la finalul Sprintului.

Subiectul 6

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. În procesul de dezvoltare Scrum, se construiește la începutul proiectului un „Product Backlog” constând din toate fragmentele de funcționalitate, adică User Stories sau scenarii, prioritizate conform nevoilor clientului. Apoi, la începutul fiecărui Sprint, se construiește un „Sprint Backlog” în care se preiau din acest „Product Backlog” cele mai prioritare fragmente de funcționalitate, exact așa cum au fost prioritizate la începutul proiectului. Dacă apreciați că afirmația este „adevărată”, explicați motivul pentru care s-ar proceda astfel în Scrum și avantajele acestei abordări; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Product Backlog-ul nu este o listă fixă, stabilită complet și definitiv la începutul proiectului. În Scrum, Product Owner-ul îl rafinează și îl reprioritizează continuu, pe baza feedbackului, valorii de business și schimbărilor de context. La Sprint Planning se alege munca potrivită pentru Sprintul curent, ținând cont de prioritatea actuală, obiectivul Sprintului și capacitatea echipei.

Subiectul 7

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. În procesul de dezvoltare Scrum, „Burndown Chart” este un grafic întocmit sau generat la sfârșitul fiecărei iterații, care arată evoluția efortului de-a lungul ultimului Sprint și care este folosit pentru planificarea Sprinturilor următoare. Dacă apreciați că afirmația este „adevărată”, amintiți detaliile din definiția „Burndown Chart” care vă susțin afirmația; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Burndown Chart-ul nu se construiește doar la sfârșitul Sprintului, ci este actualizat pe parcurs, de regulă zilnic. El arată evoluția muncii rămase în Sprintul curent, nu doar o imagine retrospectivă. Poate oferi informații utile pentru discuții viitoare, dar rolul său principal este monitorizarea progresului în Sprintul aflat în desfășurare.

Subiectul 8

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Practica de „Pair Programming” din procesele agile se referă la faptul că doi programatori lucrează în paralel la aceeași funcționalitate, dar având roluri diferite: unul scrie codul funcționalității implementate, în timp ce al doilea programator scrie codul pentru testele unitare. În acest fel, amândoi ajung să fie familiari cu funcționalitatea respectivă. Dacă apreciați că afirmația este „adevărată”, explicați ce roluri au cei doi programatori și de ce practica ajută la înțelegerea funcționalității; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Pair Programming nu înseamnă împărțirea muncii între un programator care scrie funcționalitatea și altul care scrie separat testele unitare. În forma clasică, doi programatori lucrează împreună la același cod: unul este „driver” și tastează, iar celălalt este „navigator” și revizuiește, gândește alternative și observă probleme. Rolurile se schimbă periodic, iar familiarizarea cu funcționalitatea apare tocmai din colaborarea pe același fir de lucru.

Subiectul 9

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Diagramele de clase ale aplicației sunt principalul rezultat al aplicării tehnicii CRC Cards. Apoi, pe baza diagramelor de clase, se scrie codul aferent scenariilor. În acest context, diagramele de secvență vin la sfârșit, adică se construiesc pe baza codului, pentru a documenta implementarea. Dacă apreciați că afirmația este „adevărată", explicați de ce ar fi necesară prezența codului pentru a crea diagrame de secvență; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Cardurile CRC sunt folosite pentru a descoperi clase candidate, responsabilități și colaboratori, nu pentru a produce direct și obligatoriu diagramele de clase finale. Diagramele de secvență nu trebuie să fie construite după cod, ci pot fi folosite încă din faza de analiză sau proiectare pentru a înțelege cum colaborează obiectele într-un scenariu. Codul poate confirma sau rafina aceste diagrame, dar nu este o condiție necesară pentru a le crea.

Subiectul 10

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Dacă un Use Case numit „Special" este într-o relație de tip «extend» cu un alt Use Case, numit „Base", atunci când modelăm sau descoperim clasele din cele două Use Case-uri, de exemplu folosind tehnica CRC Cards, multe dintre clasele din „Special" vor fi, de fapt, clase derivate ale claselor din „Base", pe care în Java le declarăm folosind relația „extends". De asemenea, dacă Use Case-ul „Special" are o relație de tip «include» orientată către un Use Case numit „Base", atunci asta înseamnă că multe dintre clasele rezultate din Use Case-ul „Special" vor conține obiecte din clasele descoperite din Use Case-ul „Base". Dacă apreciați că afirmația este „adevărată", explicați legătura dintre relațiile între Use Case-uri și relațiile dintre clase; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Relațiile «extend» și «include» dintre Use Case-uri descriu relații între comportamente de utilizare, nu relații de moștenire sau compoziție între clase. «extend» indică un comportament opțional sau condițional adăugat unui Use Case de bază, iar «include» indică un comportament comun reutilizat de mai multe Use Case-uri. Faptul că două Use Case-uri sunt legate astfel nu implică automat clase derivate Java sau obiecte conținute în alte obiecte.

Subiectul 11

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Relația de tip «include» între Use Case-uri este folosită pentru a reprezenta cazurile în care comportamentul unui Use Case este inclus în comportamentul mai multor Use Case-uri de bază. Dacă apreciați că afirmația este „adevărată", explicați ce comportament se reutilizează și cum se interpretează direcția relației «include»; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este adevărată. Relația «include» se folosește atunci când un Use Case de bază include în mod obligatoriu comportamentul unui alt Use Case, de obicei pentru a evita repetarea aceluiași comportament în mai multe locuri. De exemplu, mai multe scenarii pot include un pas comun de autentificare. Trebuie însă avut grijă la direcția relației: Use Case-ul de bază îl include pe cel reutilizat, nu invers.

Subiectul 12

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Fiecare card CRC ar trebui să reprezinte mai multe clase, mai exact toate clasele implicate într-un singur scenariu. Dacă apreciați că afirmația este „adevărată", explicați de ce un card CRC ar grupa toate clasele dintr-un scenariu; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Un card CRC reprezintă o singură clasă candidată. Pe card se notează numele clasei, responsabilitățile ei și clasele cu care colaborează. Un scenariu poate implica mai multe carduri CRC, pentru că mai multe clase colaborează pentru realizarea lui.

Subiectul 13

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Într-o diagramă de clase am reprezentat relația dintre clasa „Mașină” și clasa „Ușă” sub forma unei linii cu un romb alb, gol, la capătul dinspre clasa „Mașină” și cu anotarea „1..4” la capătul dinspre clasa „Ușă”. Această relație ne spune că obiectul „Ușă” va putea fi refolosit în alte obiecte „Mașină”, însă nu în mai mult de 4 obiecte „Mașină” în același timp. Dacă apreciați că afirmația este „adevărată”, explicați ce tip de relație este cea descrisă în enunț și care sunt particularitățile acestei relații; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Rombul alb la capătul dinspre „Mașină” indică o relație de agregare, adică o relație de tip întreg–parte mai slabă decât compoziția. Multiplicitatea „1..4” la capătul dinspre „Ușă” spune că o mașină are între una și patru uși. Ea nu spune că același obiect „Ușă” poate fi refolosit în până la patru obiecte „Mașină” în același timp.

Subiectul 14

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Euristicile de proiectare orientată pe obiecte ne spun că, pentru a evita problema proliferării claselor, trebuie ca întotdeauna, când modelăm o clasă „Familie”, să evităm crearea unor clase separate pentru „Mamă”, „Tată” și „Copil”, întrucât este mereu suficient să reprezentăm membrii familiei ca instanțe ale unei clase „Persoană”. Dacă apreciați că afirmația este „adevărată”, explicați de ce este suficientă reprezentarea prin instanțe ale clasei „Persoană”; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Euristica de a evita proliferarea claselor spune să nu creăm clase separate fără un motiv real, nu că trebuie să le evităm întotdeauna. Dacă „Mamă”, „Tată” și „Copil” au doar roluri diferite ale aceleiași entități, o clasă „Persoană” cu roluri sau relații poate fi suficientă. Dacă însă aceste concepte au responsabilități, reguli sau comportamente diferite în aplicație, pot fi justificate clase separate.

Subiectul 15

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Diagramele de secvență arată ordinea în care se apelează metodele unei clase, de la începutul programului până la sfârșit. Dacă apreciați că afirmația este „adevărată”, explicați cum este reprezentată ordinea apelurilor într-o diagramă de secvență; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Diagramele de secvență descriu mesajele schimbate între obiecte sau actori într-un anumit scenariu. Ele arată ordinea temporală a interacțiunilor, de sus în jos. Nu descriu toate apelurile unei singure clase de la începutul până la sfârșitul programului.

Subiectul 16

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Atât relația de agregare (aggregation), cât și cea de compoziție (composition) implică faptul că, dacă clasa de bază este ștearsă, toate clasele derivate vor fi de asemenea șterse. Dacă apreciați că afirmația este „adevărată”, explicați ce se întâmplă cu obiectele parte în cazul agregării și în cazul compoziției; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Agregarea și compoziția nu sunt relații de tip „clasă de bază – clasă derivată”, ci relații întreg–parte între obiecte. În compoziție, partea are de obicei un ciclu de viață dependent de întreg, deci distrugerea obiectului întreg poate duce la distrugerea părților. În agregare, legătura este mai slabă, iar părțile pot exista independent de întreg.

Subiectul 17

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. În stilul arhitectural Blackboard/Repository, termenul de „producători” (Producers) se referă la membrii echipei de dezvoltare care implementează baza de date și importă datele, în timp ce „consumatorii” (Consumers) sunt programatorii care implementează interfața cu utilizatorul prin care vor fi folosite sau consumate datele sistemului. Dacă apreciați că afirmația este „adevărată”, explicați rolurile de producător și consumator în acest stil arhitectural; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. În Blackboard/Repository, „producătorii” și „consumatorii” sunt componente software, nu roluri din echipa de dezvoltare. Producătorii scriu sau actualizează informații în repository/blackboard, iar consumatorii citesc sau folosesc acele informații. Aceeași componentă poate chiar să producă unele date și să consume altele, în funcție de rolul ei în proces.

Subiectul 18

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. În stilul arhitectural Layered (stratificat), un avantaj al straturilor este că fiecare strat poate fi dezvoltat de către persoane cu competențe diferite: de exemplu, stratul de prezentare poate fi implementat de programatori de frontend, logica funcțională a aplicației de programatori de backend, iar stratul de date de specialiști în baze de date. Dacă apreciați că afirmația este „adevărată", explicați cum se execută o cerere (request) venită de la utilizator prin toate straturile și de ce separarea pe straturi este utilă; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este adevărată. În stilul stratificat, împărțirea pe straturi ajută la separarea preocupărilor: interfață, logică de business și acces la date. O cerere venită de la utilizator ajunge mai întâi în stratul de prezentare, apoi este transmisă către logica aplicației, care poate apela stratul de date, iar răspunsul urcă apoi înapoi prin straturi. Avantajul nu este doar împărțirea oamenilor pe competențe, ci și faptul că fiecare strat poate fi înțeles, testat și modificat mai ușor.

Subiectul 19

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Structura stilului arhitectural Layered are ca principal avantaj performanța de timp a aplicației, întrucât fiecare strat comunică doar cu stratul imediat inferior. Dacă apreciați că afirmația este „adevărată", explicați cum se execută o cerere (request) venită de la utilizator prin toate straturile și de ce această structură ar îmbunătăți performanța; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Stilul Layered nu are ca avantaj principal performanța de timp; dimpotrivă, trecerea prin mai multe straturi poate introduce costuri suplimentare. Avantajele principale sunt separarea responsabilităților, mentenabilitatea, testabilitatea și posibilitatea de a înlocui sau modifica un strat cu impact mai mic asupra celorlalte. O cerere parcurge de obicei straturile de sus în jos, apoi rezultatul este întors în sens invers.

Subiectul 20

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. În stilul arhitectural Blackboard/Repository, implementarea unei noi componente (knowledge source) trebuie să țină cont de felul în care sunt implementate celelalte componente. Dacă apreciați că afirmația este „adevărată", explicați de ce o nouă componentă ar trebui să depindă de implementarea celorlalte knowledge sources; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. În Blackboard/Repository, o componentă nouă trebuie să respecte structura datelor și interfețele repository-ului, nu să cunoască implementarea internă a celorlalte componente. Această decuplare este chiar unul dintre avantajele stilului. Desigur, componentele trebuie să fie compatibile la nivel de date și contracte, dar nu ar trebui să depindă direct de codul sau algoritmii celorlalte knowledge sources.

Subiectul 21

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Vrem să testăm unitar (Unit Testing) o clasă „Mașină" care are în componența sa 4 obiecte de tip „Ușă" și un obiect de tip „Motor". În acest caz, este mai simplu de testat clasa atunci când toate obiectele parte, „Ușă" și „Motor", sunt deja create în constructorul clasei „Mașină", față de alternativa de a injecta din exterior, prin parametrii constructorului, obiectele parte preconstruite. Dacă apreciați că afirmația este „adevărată", explicați de ce această soluție ar fi mai bună pentru testarea unitară; iar dacă apreciați că afirmația este „falsă", explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Dacă „Mașina" își creează singură ușile și motorul în constructor, testul devine mai greu de izolat, pentru că nu putem controla ușor acele dependențe. Prin injectarea dependențelor putem folosi obiecte false, mock-uri sau variante simple ale claselor „Ușă" și „Motor". Astfel testarea unitară se concentrează mai bine pe comportamentul clasei „Mașină", nu pe comportamentul tuturor claselor asociate.

Subiectul 22

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Rolul testării de integrare este să valideze că, după ce testarea unitară a arătat că modulele luate separat funcționează corect, atunci când integrăm aceste module, fie bottom-up, fie top-down, ele funcționează corect împreună, deși fără a se putea garanta că nu există defecte. Dacă apreciați că afirmația este „adevărată”, explicați ce anume verifică testarea de integrare și ce rol au strategiile bottom-up și top-down; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este adevărată. Testarea de integrare verifică dacă module care au fost testate separat funcționează corect atunci când sunt conectate între ele. Ea poate descoperi probleme de interfață, de format al datelor, de ordine a apelurilor sau de presupuneri diferite între module. Strategiile bottom-up și top-down sunt moduri de a decide ordinea în care integrăm modulele și ce componente auxiliare folosim în timpul testării.

Subiectul 23

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Complexitatea ciclomatică (Cyclomatic Complexity) nu afectează numărul de cazuri de test necesare pentru testarea de tip black-box. Dacă apreciați că afirmația este „adevărată”, explicați de ce complexitatea ciclomatică este legată mai ales de testarea white-box și cum se aleg cazurile de test black-box; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este adevărată în sens strict metodologic. Complexitatea ciclomatică este calculată pe baza structurii interne a codului și este folosită mai ales în testarea white-box, pentru a estima numărul de căi independente. În testarea black-box, cazurile de test sunt derivate din cerințe, clase de echivalență, valori-limită și comportamentul observabil. Complexitatea internă poate sugera riscuri, dar nu determină direct numărul de cazuri black-box.

Subiectul 24

Enunț: Pentru afirmația următoare, precizați dacă aceasta este adevărată sau falsă și argumentați-vă răspunsul. Testarea de regresie (regression testing) este necesară doar atunci când se găsesc un număr mare de bug-uri în software. Dacă apreciați că afirmația este „adevărată”, explicați de ce numărul mare de bug-uri ar fi condiția principală pentru testarea de regresie; iar dacă apreciați că afirmația este „falsă”, explicați tot ce este greșit în această afirmație.

Răspuns-model: Afirmația este falsă. Testarea de regresie este necesară ori de câte ori modificăm software-ul și vrem să verificăm că funcționalitățile existente nu au fost stricate. Ea se aplică după repararea unui bug, adăugarea unei funcționalități, refactorizare sau integrarea unor componente noi. Numărul de bug-uri găsite nu este condiția principală pentru regresie; important este riscul ca o schimbare să afecteze comportamente deja corecte.